

```
#define _(String) gettext (String)
int main() {
    setlocale (LC_ALL, "");
    bindtextdomain ("helloi18n", "/usr/share/locale");
    textdomain ("helloi18n");
    printf _(_("Hello World\n"));
}
```

The internationalisation instrumentation consists of adding header files (*libintl.h* and *locale.h*), and calling three functions at the start of the program. The *setlocale* function signals to the runtime that the locale as defined in the user environment needs to be used. The *bindtextdomain* function takes a package name and directory name as arguments. For a typical software project, there will be many source files that need to be compiled as a group, to build a package. Since we are compiling just one file, I have just used the name of the source file. The directory refers to the path where the message catalogues (the set of translated messages for the program) are available. The *textdomain* function is used to associate the filename for messages used in the program. The *printf* call is modified by placing the format string inside a function *_()*. The preprocessor will generate the *gettext* function calls. The localisation software *xgettext* (explained later) uses this as a keyword to extract translatable messages.

Now, let us compile and run the program:

```
$ gcc -o helloi18n helloi18n.c
Hello world
```

Localisation

Let us extract the strings, translate them, and prepare a binary so that the program can use another locale, using the *Gettext* library programs.

```
$ xgettext -k _ --package-name=helloi18n --package-version=1.0 -o
helloi18n.pot helloi18n.c
```

The parameter *-k _* specifies that we are interested in messages that are part of the function call with the underscore name. The package and version names are used in the header information. Usually, these are specified as part of the build files. As our example is fairly simple, I have used command-line arguments to pass these names. The option *-o* helps specify the file type as portable object template (pot) rather than the default portable object (po).

Listing 3: helloi18n.pot

```
# SOME DESCRIPTIVE TITLE.
# Copyright (C) YEAR THE PACKAGE'S COPYRIGHT HOLDER
# This file is distributed under the same license as the PACKAGE
package.
# FIRST AUTHOR <EMAIL@ADDRESS>, YEAR.
#
```

```
#, fuzzy
msgid ""
msgstr ""
"Project-Id-Version: helloi18n 1.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2012-06-04 22:43+0530\n"
"PO-Revision-Date: YEAR-MO-DA HO:MI+ZONE\n"
"Last-Translator: FULL NAME <EMAIL@ADDRESS>\n"
"Language-Team: LANGUAGE <LI@li.org>\n"
"Language: \n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=CHARSET\n"
"Content-Transfer-Encoding: 8bit\n"

#: helloi18n.c:15
#, c-format
msgid "Hello World\n"
msgstr ""
```

Listing 3 shows the *pot* file for our example program. It consists of a few header lines of information, followed by the source string and translated string, prefixed with *msgid* and *msgstr*, respectively. The first message is designed to give general information about the translation, such as the date, the name of the translator and team, etc. The subsequent lines provide messages used in the program. Each message follows the syntax as given in Listing 4.

Listing 4: Syntax for messages in a .po file.

```
white-space
# translator-comments
#. extracted-comments
#: reference...
#, flag...
#| msgid previous-untranslated-string
msgid untranslated-string
msgstr translated-string
```

As the meaning of a word could be different based on the context, the above format provides options for the translator and programmer to provide additional context information. The file name and line number at which the message appears is provided as part of the reference. The flag is used to indicate the status or specific nature of translation. If it is fuzzy, it means that the translator needs to confirm the translation, as it was guessed from previous translation data. If it is obsolete, it could mean that it can be removed from the file. If it is C-format, it is an alert to the translator that this message is used in input or output statements, and follows the C language format and can have format information, newline information, etc. The previous-untranslated-string is updated by the *msgmerge* program (not described in this article), when creating a new pot file using previous translation files. Let's use *helloi18n.pot* and a version for the Telugu language with blank translations.